

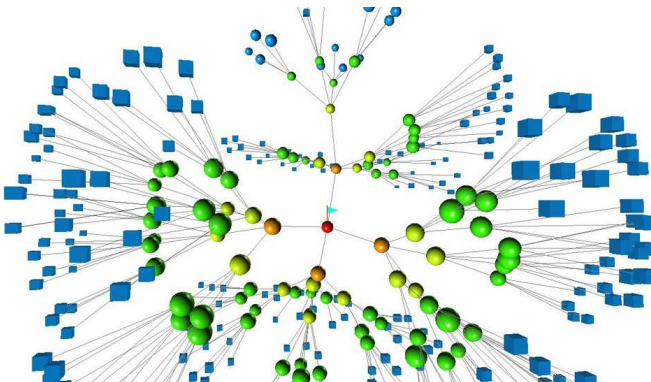
Introduction to Data Analysis using R

Eslam Montaser Roushdi
Facultad de Informática
Universidad Complutense de Madrid
Grupo G-Tec UCM
www.tecnologiaUCM.es

February, 2014

Our aim

- Study and describe in depth analysis of Big Data by using the R program and learn how to explore datasets to extract insight.



Outlines:

- 1 Getting Started - R Console.
- 2 Data types and Structures.
- 3 Exploring and Visualizing Data.
- 4 Programming Structures and Data Relationships.

1)Getting Started - R Console.



R program: is a free software environment for data analysis and graphics.

- R program:
 - i) **Programming language.**
 - ii) **Data analysis tool.**
- R is used across many industries such as healthcare, retail, and financial services.
- R can be used to analyze both structured and unstructured datasets.
- R can help you explore a new dataset and perform descriptive analysis.



1) Getting Started - R Console.

R – Console (as a calculator)

Expressions

Assignment operator (don't use =)

```
>a <- 5  
>b <- 4  
>a + b  
[1] 9  
>ls()  
[1] "a" "b"  
>typeof(a)  
[1] "double"
```

Set values to variable / object

Math Operation

Displayed output

List active variables

Retrieve data type

2) Data types and Structures.

i) Data types.

numeric, **logical**, and **character** data types.

Data types

real

```
> c <- 75.3
```

```
> typeof(c)
```

```
[1] "double"
```

integer

```
> b <- as.integer(8)
```

```
> typeof(b)
```

```
[1] "integer"
```

logical

```
> is.integer(b)
```

```
TRUE
```

character

```
> typeof(grade <- "B")
```

```
[1] "character"
```

2) Data types and Structures.

ii) Data structures.

- Vector.
- List.
- Multi-Dimensional (Matrix/Array - Data frame).

R Vectors

- A series of numbers
- Created with
 - `c()` to concatenate elements or sub-vectors
 - `rep()` to repeat elements or patterns
 - `seq()` or `m:n` to generate sequences
- Most mathematical functions and operators can be applied to vectors
 - Without loops!

2) Data types and Structures.

Defining Vectors

```
> rep(1,10)           # repeats the number 1, 10 times
[1] 1 1 1 1 1 1 1 1 1 1
> seq(2,6)            # sequence of integers between 2 and 6
[1] 2 3 4 5 6          # equivalent to 2:6
> seq(4,20,by=4)      # Every 4th integer between 4 and 20
[1] 4 8 12 16 20
> x <- c(2,0,0,4)     # Creates vector with elements 2,0,0,4
> y <- c(1,9,9,9)
> x + y               # Sums elements of two vectors
[1] 3 9 9 13
> x * 4               # Multiplies elements
[1] 8 0 0 16
> sqrt(x)             # Function applies to each element
[1] 1.41 0.00 0.00 2.00 # Returns vector
```

2) Data types and Structures.

Lists

Collection of vectors

- **mixed** data types
- varying length objects are allowed

```
> students <- c("Mary", "Bob", "John")
```

```
> ages <- c(14, 15, 18)
```

```
> classroom <- list(students, ages)
```



2) Data types and Structures.

Matrix – creation and access

Matrix - Two (2) dimensional vector

```
> scores <- matrix(data = c(1, 2, 80, 85, 67, 56),  
  nrow = 2, ncol = 3)
```



```
> scores
```

	[, 1]	[, 2]	[, 3]
[1,]	1	80	67
[2,]	2	85	56

Fill by column (default)

```
> mean(scores[, 2])
```

```
[1] 82.5
```



Slice by row or column index

2) Data types and Structures.

Matrix – naming columns

Matrix access using names

```
> scores
```

	[, 1]	[, 2]	[, 3]
[1,]	1	80	67
[2,]	2	85	56

```
> colnames(scores) <- c("id", "test1", "test2")
```

```
> scores
```

	id	test1	test2
[1,]	1	80	67
[2,]	2	85	56

```
> mean(scores[, "test1"])
```

```
[1] 82.5
```

Slice by column name



2) Data types and Structures.

Data Frames

Data Frame

- mixed data types
- defined size (# rows, # columns)

```
> df1 <- data.frame(students = c("Mary", "Jane", "Eva"),  
  test1 = c(80, 90, 70))
```

```
> df1
```

	students	test1
1	Mary	80
2	Jane	90
3	Eva	70



2) Data types and Structures.

Note that

- Adding columns of data.
`df1 <- cbind (df1, The new column).`
- Adding rows of data.
`df1 <- rbind (df1, The new row).`

Missing Data

Large datasets often have missing data.

- Most R functions can handle.

```
> ages <- c (23, 45, NA)  
> mean(ages)  
[1] NA  
> mean(ages, na.rm=TRUE)  
[1] 34
```
- Where, **NA** is a logical constant of length 1 which contains a missing value indicator.

3) Exploring and Visualizing Data.

- **Importing and Exporting data.**
- **Filtering/Subsets.**
- **Sorting.**
- **Visualization/Analysis data.**

How to import external data from files into R?

Reading Data from text files:

- **Multiple functions to read in data from text files.**
- **Types of Data formats.**
 - **Delimited.**
 - **positional.**



3) Exploring and Visualizing Data.

Reading external data into R

Delimited files

R includes a family of functions for importing delimited text files into R, based on the **read.table** function:

```
read.table(file, header, sep = , quote = , dec = , row.names, col.names,  
as.is = , na.strings , colClasses , nrow = , skip = , check.names = ,  
fill = , strip.white = , blank.lines.skip = , comment.char = ,  
allowEscapes = , flush = , stringsAsFactors = , encoding = )
```

For example

```
name.last,name.first,team,position,salary  
"Manning","Peyton","Colts","QB",18700000  
"Brady","Tom","Patriots","QB",14626720  
"Pepper","Julius","Panthers","DE",14137500  
"Palmer","Carson","Bengals","QB",13980000  
"Manning","Eli","Giants","QB",12916666
```



3) Exploring and Visualizing Data.

Note that

- The first row contains the column names.
- Each text field is encapsulated in quotes.
- Each field is separated by commas.

How to load this file into R

the first row contained column names (**header=TRUE**), that the delimiter was a comma (**sep=","**), and that quotes were used to encapsulate text (**quote="\\""**).

The R statement that loads in this file:

```
> top.5.salaries <- read.table("top.5.salaries.csv",  
+ header=TRUE,  
+ sep="," ,  
+ quote="\\"")
```



3) Exploring and Visualizing Data.

Fixed-width files

To read a fixed-width format text file into a data frame, you can use the **read.fwf** function:

```
read.fwf(file, widths, header = , sep = , skip = , row.names, col.names,  
n = , bufferize = ,...)
```

Note that

read.fwf can also take many arguments used by `read.table`, including `as.is`, `na.strings`, `colClasses`, and `strip.white`.

3) Exploring and Visualizing Data.

Let's explore a public data using R.

Exploring data – importing

Names example

– Contents of the CSV file

```
Ontario Top Baby Names (Male),,  
Year,Name,Frequency  
1999,AADAM,5  
2005,AADAM,6  
2007,AADEN,10
```

File attributes

- approximately 66 thousand records in the file
- skip first line
- use second line as the header (column names)

```
> n <- read.csv("males.csv", skip=1, header=TRUE)
```



3) Exploring and Visualizing Data.

Exploring data – post import

Data Verification - post import

>str(n) # structure of n

```
> str(n)
'data.frame': 66351 obs. of  3 variables:
 $ Year      : int  1999 2005 2007 2008 2009 2010
 $ Name      : Factor w/ 3736 levels "AADAM","AAD
 3 3 ...
 $ Frequency: int   5 6 10 34 38 12 10 6 10 5 ...
```

>dim(n) # rows and columns

```
> dim(n)
[1] 66351      3
```

as.is=TRUE
stringsAsFactors=FALSE



3) Exploring and Visualizing Data.

Data Exploration - subsets

Examine most recent data - bracket [] notation

- Find the most recent year

```
>max(n$Year)
```

```
2010
```

Goal : Find the 5 most popular names in the most recent year.

- Create a new data frame with only 2010 data

```
>n.2010<-n[n$Year==2010,]
```

```
>nrow(n.2010)
```

```
[1] 1503
```

Filter by row
(all columns)

3) Exploring and Visualizing Data.

Data Exploration – sorting vectors

Sorting data in vectors

```
>ages <- c(78,45,23,12)
```

```
>sort(ages)
```

```
[1] 12 23 45 78
```

```
>sort(ages, decreasing=TRUE)
```

```
[1] 78 45 23 12
```



3) Exploring and Visualizing Data.

Data Exploration – sorting data frames

Sorting data in data frames

- Remove the Year column from the (*Projection*)

```
>n.2010<-n.2010[,c("Name", "Frequency")]
```

```
>head(n.2010,2)
```

	Name	Frequency
6	AADEN	12
17	AAHIL	8

Filter by Columns
(all rows)

- Sort the data using `order()`

```
> n.2010.s <-n.2010[order(n.2010$Frequency, decreasing=TRUE),]
```

```
> head(as.character(n.2010.s$Name), 5)
```

```
[1] "ETHAN" "LIAM" "JACOB" "LUCAS" "NOAH"
```

3) Exploring and Visualizing Data.

Now let's visualize trends in our data using Data Visualizations or graphics

Data Visualization

Goal : Show trends from 1917 to 2010 for Names

1. Ask the user for a name to explore
2. Build a table of frequencies of registrations by year
3. Create a visualization (chart)



3) Exploring and Visualizing Data.

Data Visualization - Trends

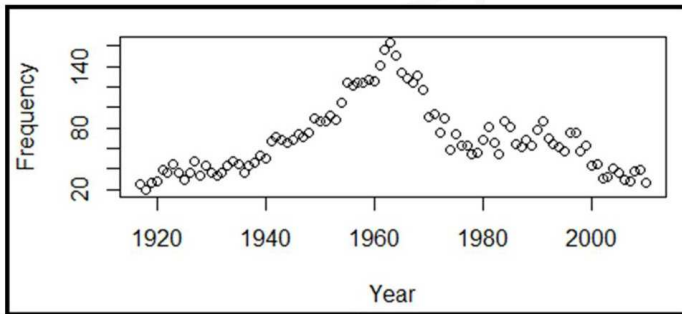
```
> #Obtain the name from the user
> name.in <- readline(prompt="Male baby name? :")
> #create data frame with frequencies and years
> freq.year<-n[n$Name == toupper(name.in),
  c("Year","Frequency")]

> #Ensure the data is sorted by year
> freq.year<-freq.year[order(freq.year$Year),]

> #Perform a simple scatterplot
> plot(freq.year)
```

3) Exploring and Visualizing Data.

Scatter Plot - default



3) Exploring and Visualizing Data.

Data Visualization – plot() function

Plot function is extremely versatile

Options	Description
main	chart title (character string)
xlab	x-axis label (character string)
ylab	y-axis label (character string)
type	Type of plot 'p' - points (default) 'l' - lines 's' - stairs



4) Programming Structures and Data Relationships.

■ Programming Structures

- Control Flow
 - decisions and iteration

- Functions

■ Multivariable Data Relationships

- Covariance
- Correlation
- Linear Regression

4) Programming Structures and Data Relationships.

Let's examine decision making in R

Control Flow - Decisions

Conditional statements

`if (cond) expr1 else if exp2 ... else expr2`

```
1 #Generate a random integer number between 1 and 100
2 set.seed(1)
3 num = as.integer(runif(1,1,100))
4
5 if (num%%2 == 0) {
6   cat(num, "is an even number.")
7 } else {
8   cat(num, "is an odd number.")
9 }
```

27 is an odd number.

4) Programming Structures and Data Relationships.

Iteration - loops

Three different methods of iterating or looping:

1. Continual `repeat`
2. Conditional `while`
3. Counted `for`

4) Programming Structures and Data Relationships.

Loops – for

Counted Loops

```
for (var in list) { expressions }
```

```
1 for (i in seq(from=5, to=15, by=5))  
2   print (i)
```

```
[1] 5  
[2] 10  
[3] 15
```

```
1 marks <- c(70,56,78,34)  
2 p <- 0  
3 for (mark in marks) {  
4   if (mark >= 50) p<-p+1  
5 }  
6 cat(p, "students passed.\n")
```

```
3 students passed.
```



4) Programming Structures and Data Relationships.

Functions - Overview

Functions – Named group of R expressions

```
function (arguments) { body }
```

- arguments
 - passed by value
 - default values
 - variable length set of input arguments allowed (...)
 - can be other functions
- body
 - contained within brace brackets {}
- variables in functions
 - local to the function
- output
 - explicit using: `return(<value>)`
 - last evaluated expression



4) Programming Structures and Data Relationships.

Functions - Example

```
> f1 <- function(a,b) { return(a+b) }  
> f2 <- function(a,b) { return(a-b) }  
> f <- f1  
> f(3,8)  
[1] 11  
> f <- f2  
> f(5,4)  
[1] 1
```

The apply family of functions

- **apply()** can apply a function to elements of a matrix or an array.
- **lapply()** applies a function to each column of a dataframe and returns a list.
- **sapply()** is similar but the output is simplified. It may be a vector or a matrix depending on the function.
- **tapply()** applies the function for each level of a factor.

4) Programming Structures and Data Relationships.

Functions – *apply()

***apply()** – Apply a function to a data structure

```
>m <- data.frame (Math = c(34,78,89),  
  Science = c(78,89,90), English = c(78,88,85))
```

```
>lapply(m, mean)
```

```
>$Math
```

```
>[1] 67
```

```
>$Science
```

```
>[1] 85.66667
```

```
>$English
```

```
>[1] 83.66667
```

	Math	Science	English
1	34	78	78
2	78	89	88
3	89	90	85

```
> means <- round(sapply(m, mean),1)  
> print(means)  
  Math Science English  
 67.0    85.7    83.7  
> means["Math"]  
Math  
67
```

4) Programming Structures and Data Relationships.

Common useful built-in functions

- `all()` #returns TRUE if all values are TRUE.
- `any()` # returns TRUE if any values are TRUE.
- `args()` # information on the arguments to a function.
- `cat()` # prints multiple objects, one after the other.
- `cumprod()` # cumulative product.
- `cumsum()` # cumulative sum.
- `mean()` # mean of the elements of a vector.
- `median()` # median of the elements of a vector.
- `order()` # prints a single R object.

4) Programming Structures and Data Relationships.

Data Relationships – Case Study

Case Study

⇒ Is there a relationship between a person's **height** and their **shoe size** ?

⇒ How do we accomplish this task ?

4) Programming Structures and Data Relationships.

Covariance

Covariance

– measure of variability between 2 variables

positive

- variables change in the same direction

eg. when heights increase the shoe size also increases

negative

- variables change in opposite directions

eg. when heights increase the shoe size decreases



4) Programming Structures and Data Relationships.

Correlation

Correlation

- degree of the linear relationship between variables

**Correlation is expressed as a number
between -1 and 1**

Value	Description
>-1 and <0	negative correlation var 1 $\uparrow$ var 2 $\downarrow$ OR var 1 $\downarrow$ var 2 $\uparrow$
0	no relationship as values of variables vary
>0 and <1	positive correlation var 1 $\uparrow$ var 2 $\uparrow$ OR var 1 $\downarrow$ var 2 $\downarrow$



4) Programming Structures and Data Relationships.

R Analysis – Generate data and coefficients

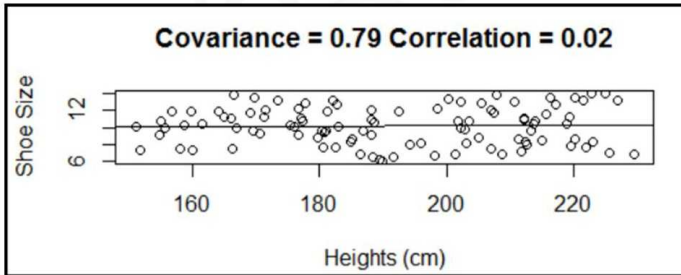
```
1 #set the randomization seed for consistency
2 set.seed(1)
3 #uniform distribution of heights
4 height <- runif(100, 150, 230)
5
6 #uniform distribution of shoe sizes
7 shoe.size <- runif(100,6,14)
8
9 #determine covariance
10 cov1<-cov(as.shoe.size,height)
11
12 #determine correlation
13 cor1<-cor(shoe.size,height)
```



4) Programming Structures and Data Relationships.

R Analysis

```
15 #create a plot of heights vs. shoe sizes
16 msg <- sprintf("Covariance = %.2f Correlation = %.2f", cov1,cor1 )
17 plot (height,shoe.size, main=msg,
18       xlab = "Heights (cm)",
19       ylab = "Shoe Size")
20
21 #create a linear regression model and plot
22 lm1 <- lm (shoe.size~height)
23 abline(lm1)
```



Thanks!!

References



Grant Hutchison, *Introduction to Data Analysis using R*, October 2013.



John Maindonald, W. John Braun, *Data Analysis and Graphics Using R: An Example-Based Approach (Cambridge Series in Statistical and Probabilistic Mathematics)*, Third Edition, Cambridge University Press 2003.



Nicholas J. Horton, Ken Kleinman, *Using R for Data Management, Statistical Analysis, and Graphics*, CRC Press, 2010.